Modeling of an Algae Pond

Octave Litrico

January 2026

Introduction

In the context of modeling an algae cultivation pond, the evolution of a scalar quantity $\rho(x,t)$, representing the concentration of algae can be described by a partial differential equation of the advection-diffusion type:

$$\frac{\partial \rho}{\partial t} + a\frac{\partial \rho}{\partial x} = \mu \frac{\partial^2 \rho}{\partial x^2}$$

where:

- a is the advection (transport) coefficient related to the movement induced by the water wheel
- μ is the diffusion coefficient (natural or turbulent dispersion).

However, to simplify the numerical analysis and better understand diffusion phenomena alone, we choose in this work to neglect the advection term. This amounts to considering only a pure diffusion equation:

$$\frac{\partial \rho}{\partial t} = \mu \frac{\partial^2 \rho}{\partial x^2}$$

1 The discretized Model

1.1 Discretization principle

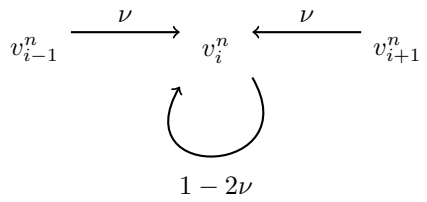
Let us discretize the time and space intervals. Let $N, M \in \mathbb{N}$, $h = \Delta_x = \frac{L}{N}$, $\Delta_t = \frac{T}{M}$ and $x_i = i\Delta_x$, $t_n = n\Delta_t$ $\forall i \in \{0, \cdots, N\}, n \in \{0, \cdots, M\}$.

The pond will be modeled by a rectangle of size $L \times H$ divided into I horizontal layers of same size. The pond is circular, therefore the concentration functions of each layers will have to be periodic with period L. A water wheel located on the x-coordinate 0 stirs the water and will be modeled by a matrix P. It will intervene in the computing of the values of the algae concentration near 0. So that there is no production nor loss of matter, P must be column-stochastic. Indeed, we have that for all $i, j \in \{1, \cdots, I\}$, $P_{i,j}$ is the amount of matter in line j sent to line i. Therefore, we must have that for all $j \in \{1, \cdots, I\}$, $\sum_{k=1}^{I} P_{k,j} = 1$

The concentration function ρ will be an element of $\mathcal{C}^1([0,T], H^1([0,L])^I)$. At each time $t_n = n\Delta t$, the concentration in cell j is denoted $\rho_j^n = \rho(t_n, x_j)$.

1.2 Cellular automat

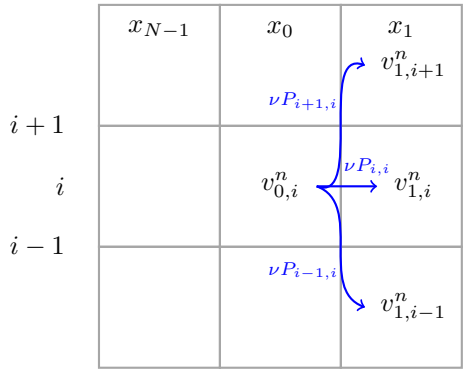
To first understand the underlying motion of the pond, let us introduce a cellular model. With the discretization showed above, we consider a cell in which the algae concentration evolves as a particle. A cell located far from the paddle wheel exchanges mass with its two neighbors. The scheme can be represented as follows:



Here, the value at time $n+1$ is a weighted linear combination of the cell and its neighbors. ν here represents the mass proportion that each cell exchanges.

1.3 Special case: near the paddle wheel

Near the paddle wheel, the stir influences the concentrations. The following scheme represents the evolution, with the wheel located in column 0 :



Therefore we end up with this automat :

$$\forall n \in \{0, \cdots, M-1\}$$

$$\begin{cases} v_j^{n+1} = \nu(v_{j-1}^n + v_{j+1}^n) + (1-2\nu)v_j^n \quad \forall j \in \{2, \cdots, N-2\} \\ v_{N-1}^{n+1} = \nu(v_{N-2}^n + Pv_0^n) + (1-2\nu)v_{N-1}^n \\ v_0^{n+1} = \nu(v_{N-1}^n + v_1^n) + (1-2\nu)Pv_0^n \\ v_1^{n+1} = \nu(Pv_0^n + v_2^n) + (1-2\nu)v_1^n \end{cases}$$

2 PDE analysis

Let $\rho(t,x) \in \mathbb{R}^I$ be the vector of algae concentration on the x-coordinate of the pond at time t. We have to take into account the $L-$periodicity :

$$\rho(t,0) = \rho(t,L) \ \forall t$$

Furthermore, the total algae mass in the pond is constant through time. Therefore, we have :

$$\sum_i \int_0^L \rho_i = 0$$

up to a translation of the values of the ρ_i by the average concentration.

Let us denote U the set :

$$\{v \in H^2([0,L])^I \ | v(0) = v(L) = Pv(L) \text{ and } \sum_{i=1}^{I} (\partial_x v_i(L) - \partial_x v_i(L)) = 0\}$$

and

$$V \text{ an eigen vector of } P \text{ such that } PV = V \text{ and } \|V\|_1 = 1$$

The ponctual values of $\partial_x u$ are well defined. Indeed, thanks to the Morrey injection, we have that $H^2([0,L]) \hookrightarrow \mathcal{C}^1([0,L])$. U is a Hilbert space for the following inner product :

$$< u, v >_{U,U} = \sum_{i=1}^{I} \frac{1}{V_i} \int_0^L \partial_x u_i . \partial_x v_i$$

The problem we are looking to solve is :

$$\begin{cases} \text{Find } \rho \in U \\ \text{such that } \partial_t \rho = \mu \Delta \rho \end{cases}$$

Firstly, let us notice that :

$$\begin{aligned}
&\forall v \in U, \\
&\langle -\Delta \rho, v \rangle_{U,U} = -\sum_{i=1}^{I} \frac{1}{V_i} \int_0^L \partial_{xx} \rho_i v \\
&= -\sum_{i=1}^{I} \frac{1}{V_i} \left[(\partial_x \rho_i(0) v_i(0) - \partial_x \rho_i(L) v_i(L)) - \int_0^L \partial_x \rho_i \partial_x v_i \right] \\
&= -\sum_{i=1}^{I} \left(\partial_x \rho_i(0) - \partial_x \rho_i(L) - \int_0^L \partial_x \rho_i \partial_x v_i \right) \quad \text{because } v_i(0) = v_i(L) = V_i \\
&= \sum_{i=1}^{I} \int_0^L \partial_x \rho_i \partial_x v_i
\end{aligned}$$

We would like to find a Hilbert basis of U which will ease our difficulties. Let us introduce $H = \{h \in L^2([0,L])^I \mid \sum_i \int_0^L h_i = 0\}$, we can easily show that U is dense in H and by using the Rellich theorem, it follows that the embedding of U in H is compact.

Let us define for all $u, v \in U$:

$$a(u,v) = \mu \sum_{i=1}^{I} \int_0^L \partial_x u_i . \partial_x v_i$$

The operator a is symmetric, bilinear, and coercive on U by the Poincaré inequality. Moreover, it is continuous. Therefore, there exists an infinite sequence $(\lambda_n)_n \in \mathbb{N}$ of positive eigenvalues of this operator, and a Hilbert basis $(x \mapsto \rho_n(x))_{n \in \mathbb{N}} \in U^{\mathbb{N}}$ of H (for the inner product $< ., . >_{H,H}$) of associated eigenvectors. Let's name them $(\rho_n)_{n \in \mathbb{N}}$. The theorem also holds that the sequence $(\frac{\rho_n}{\sqrt{\lambda_n}})$ is a Hilbert basis of U for the inner product $a(.,.)$.

If ρ is solution to (1) then there exists a sequence $(t \mapsto \nu_n(t))_{n \in \mathbb{N}}$ of functions of $H^2([0,T])$ such that:

$$\rho(t,x) = \sum_{\mathbb{N}} \nu_n(t) \rho_n(x)$$

The values of ρ_n and of ν_n must be found.

$$-\mu \Delta u = \lambda u$$

with $\lambda > 0$ So

$$u(x) = A \cos\left(\sqrt{\frac{\lambda}{\mu}} x \right) + B \sin\left(\sqrt{\frac{\lambda}{\mu}} x \right)$$

with $A, B \in \mathbb{R}^I$

Moreover, we know that $u(0) = Pu(0)$ and that P is irreducible and thus has an eigenspace associated to the eigenvalue 1 of dimension 1. Let us denote V a vector of norm 1 of this space. As each V_i is positive (given the ergodic theorem) we have $\|V\|_1 = \sum_i V_i = 1$ Then, $A = \alpha V$ with $\alpha \in \mathbb{R}$

With $u_i(0) = u_i(L)$ we see that :

$$B \sin\left(\sqrt{\frac{\lambda}{\mu}}L\right) = A\left[1 - \cos\left(\sqrt{\frac{\lambda}{\mu}}L\right)\right]$$

If $\sqrt{\frac{\lambda}{\mu}}L \notin \pi\mathbb{Z}$ then :

$$B = \alpha V \frac{(1 - \cos(\sqrt{\frac{\lambda}{\mu}}L))}{\sin(\sqrt{\frac{\lambda}{\mu}}L)}$$

Because $\sum_i \int_0^L \rho_i = 0$we have :

$$\sum_i \left[A_i \sin\left(\sqrt{\frac{\lambda}{\mu}}L\right) + B_i\left(1 - \cos\left(\sqrt{\frac{\lambda}{\mu}}L\right)\right)\right] = 0$$

i.e

$$\alpha\left(1 - \cos\left(\sqrt{\frac{\lambda}{\mu}}L\right)\right)^2 = -\alpha \sin\left(\sqrt{\frac{\lambda}{\mu}}L\right)^2$$

therefore, as $\alpha \neq 0$ because $u \neq 0$, we have :

$$(1 - \cos(\sqrt{\frac{\lambda}{\mu}}L))^2 = -(1 - \cos(\sqrt{\frac{\lambda}{\mu}}L)^2)$$

We end up with the following equation :

$$\cos\left(\sqrt{\frac{\lambda}{\mu}}L\right) = 1$$

i.e $\sqrt{\frac{\lambda}{\mu}}L \in 2\pi\mathbb{Z}$, contradiction.

Therefore, $\sqrt{\frac{\lambda}{\mu}}L \in \mathbb{N}^*\pi$

$\sqrt{\frac{\lambda}{\mu}}L = n\pi$ with $n \in \mathbb{Z}^*$.

- If $n \in 2\mathbb{N}^*$ then the equation $\sum_i \int_0^L \rho_i = 0$ is trivial, and there is no constraint on the B_i.
- If $n \in 2\mathbb{N}^* + 1$ then we have :

$$\sum_i B_i = 0$$

and :

$$A(1 - \cos(\sqrt{\frac{\lambda}{\mu}}L)) = 2A = 0$$

so

$$\alpha = 0$$

We have eventually found all the eigen vectors of the operator a. These are the elements of the following sets :

- $\{x \mapsto V\cos(\frac{2n\pi x}{L})\}_{n\in\mathbb{N}^*}$
- $\{x \mapsto e_1\sin(\frac{2n\pi x}{L})\}_{n\in\mathbb{N}^*}$
- $\cdots$
- $\{x \mapsto e_I\sin(\frac{2n\pi x}{L})\}_{n\in\mathbb{N}^*}$
- $\left\{x \mapsto \sum_{i=1}^{I}\gamma_{2n+1}^i e_i\sin(\frac{(2n+1)\pi x}{L}) \;\middle|\; \sum_{i=1}^{I}\gamma_{2n+1}^i = 0\right\}_{n\in\mathbb{N}^*}$

As shown before, these vectors form a Hilbert basis of U equipped with the inner product $a(.,.)$ up to normalization.Therefore we can decompose ρ in this basis : there exist $\mathcal{C}^1([0,T],\mathbb{R})$ functions $\alpha_{2n}, \beta_{2n}^i, \gamma_{2n+1}^i \forall n \in \mathbb{N}^*, i \in \{1,\cdots,I\}$ such that

$$\rho(x,t) = \sum_{n\in\mathbb{N}^*}\left[\alpha_{2n}(t)V\cos\left(\frac{2n\pi x}{L}\right) + \sum_{i=1}^{I}\beta_{2n}^i(t)e_i\sin\left(\frac{2n\pi x}{L}\right) + \sum_{i=1}^{I}\gamma_{2n+1}^i(t)e_i\sin\left(\frac{(2n+1)\pi x}{L}\right)\right]$$

with $\sum_{i=1}^{I}\gamma_{2n+1}^i(t) = 0 \;\forall t$

Therefore for all $n \in \mathbb{N}^*, i \in \{1,\cdots,I\}$, $\alpha_{2n}, \beta_{2n}^i, \gamma_{2n+1}^i$ satisfy the respective equations

$$\begin{cases} \alpha_{2n}'(t) = \frac{-4n^2\pi^2\mu}{L^2}\alpha_{2n}(t) \\ {\beta_{2n}^i}'(t) = \frac{-4n^2\pi^2\mu}{L^2}\beta_{2n}^i(t) \\ {\gamma_{2n+1}^i}'(t) = \frac{-(2n+1)^2\pi^2\mu}{L^2}\gamma_{2n+1}^i(t) \end{cases} \quad \text{for all } t \in [0,T]$$

Therefore,

$$\begin{cases} \alpha_{2n}(t) = \alpha_{2n}(0)\, e^{\frac{-4n^2\pi^2\mu}{L^2}t} \\ \beta_{2n}^i(t) = \beta_{2n}^i(0)\, e^{\frac{-4n^2\pi^2\mu}{L^2}t} \\ \gamma_{2n+1}^i(t) = \gamma_{2n+1}^i(0)\, e^{\frac{-(2n+1)^2\pi^2\mu}{L^2}t} \end{cases} \quad \text{for all } t \in [0,T]$$

Furthermore, as for all $n \in \mathbb{N}^*, i \in \{1,\cdots,I\}$ we have :

$$\rho_0(x) = \sum_{n\in\mathbb{N}^*}\left[\alpha_{2n}(0)V\cos(\frac{2n\pi x}{L}) + \sum_{i=1}^{I}\beta_{2n}^i(0)e_i\sin(\frac{2n\pi x}{L}) + \sum_{i=1}^{I}\gamma_{2n+1}^i(0)e_i\sin(\frac{(2n+1)\pi x}{L})\right]$$

with for all $n \in \mathbb{N}^*$:

$$\begin{cases} \alpha_{2n}(0) = \frac{2}{L\|V\|_2^2}\int_0^L \rho_0(x)\cdot V\cos(\frac{2n\pi x}{L})dx & \text{for a certain } V_j \neq 0 \\ \beta_{2n}^i(0) = \frac{2}{L}\int_0^L \rho_0(x)\cdot e_i\sin(\frac{2n\pi x}{L})dx \\ \gamma_{2n+1}^i(0) = \frac{2}{L}\int_0^L \rho_0(x)\cdot e_i\sin(\frac{(2n+1)\pi x}{L})dx \end{cases}$$

The coefficients α, β and γ are imposed by the initial condition. We have therefore found the unique solution to (1).

3 Convergence of the cellular model

3.1 Numerical scheme of the PDE

We are looking for an approximation of the exact solution ρ. We denote $\rho_j^n = \rho(t_n, x_j)$ and $\rho_h^n = (\rho_0^n, \rho_1^n, \ldots, \rho_{N-1}^n)^T$

In this section, we impose that P must be irreducible, so that there exists a unique vector V such that :

$$\|V\|_1 = 1, V_i \geq 0 \ \forall i \text{ and } PV = V$$

Using the expression of the discrete Laplacian and time derivative, we end up with this scheme :

$$\begin{cases} \dfrac{v_j^{n+1} - v_j^n}{\Delta_t} - \mu \dfrac{v_{j+1}^n - 2v_j^n + v_{j-1}^n}{\Delta_x^2} = 0, & \forall j \in \{1, \dots, N-1\},\ n \in \{0, \dots, M-1\}, \\ \displaystyle\sum_{i=1}^{I} \left(\frac{v_{1,i}^n - v_{0,i}^n}{\Delta_x} - \frac{v_{0,i}^n - v_{N-1,i}^n}{\Delta_x} \right) = 0, & \forall n \in \{0, \dots, M-1\}, \\ v_0^n = v_N^n, & \forall n \in \{0, \dots, M\}, \\ Pv_0^n = v_0^n, & \forall n \in \{0, \dots, M\}. \end{cases}$$

We can show that it is equivalent to :

$\forall n \in \{0, \cdots, M-1\}$

$$\begin{cases} v_j^{n+1} = \nu(v_{j-1}^n + v_{j+1}^n) + (1-2\nu)v_j^n \quad \forall j \in \{1, \cdots, N-2\} \\ v_{N-1}^{n+1} = \nu(v_{N-2}^n + v_0^n) + (1-2\nu)v_{N-1}^n \\ v_0^n = \alpha^n V \text{ with } \alpha^n = \frac{1}{2}(\sum_{i=1}^{I} v_{1,i}^n + v_{N-1,i}^n) \end{cases}$$

3.2 Consistency, Stability and Convergence Analysis

We now show that the cellular model defined in the previous section is consistent and stable in the discrete L^2 norm, and therefore convergent toward the continuous diffusion equation.

Discrete notation. For each time index n, we collect the discrete unknowns in a vector

$$v_h^n = (v_0^n, v_1^n, \dots, v_{N-1}^n)^T \in \mathbb{R}^{IN},$$

where each component $v_j^n \in \mathbb{R}^I$ represents the concentrations of the I layers at node x_j. The discrete L^2 inner product and norm are defined by

$$(u_h, v_h)_h = \Delta_x \sum_{j=0}^{N-1} (u_j^T v_j), \qquad \|u_h\|_h = (u_h, u_h)_h^{1/2}.$$

Matrix form of the scheme. The fully discrete explicit scheme can be written compactly as

$$v_h^{n+1} = Q v_h^n, \tag{1}$$

where Q is a block matrix of size $IN \times IN$ that includes:

- the tridiagonal coupling due to horizontal diffusion with coefficients $(\nu, 1-2\nu, \nu)$,
- the boundary connections involving the matrix P.

Consistency. We haven't shown the constistency at this point. Let ρ be a sufficiently exact solution of the continuous diffusion equation. We insert ρ into the discrete scheme to obtain the local truncation (consistency) error:

$$\tau_j^n = \frac{\rho(x_j, t_{n+1}) - \rho(x_j, t_n)}{\Delta_t} - \mu \frac{\rho(x_{j+1}, t_n) - 2\rho(x_j, t_n) + \rho(x_{j-1}, t_n)}{\Delta_x^2}, \qquad j = 2, \dots, N-2.$$

For the interior nodes, the Taylor expansion of ρ in t and x around (x_j, t_n) gives

$$\tau_j^n = \frac{\Delta_t}{2}\,\partial_{tt}\rho(x_j,t_n) - \frac{\mu\,\Delta_x^2}{12}\,\partial_{x^4}\rho(x_j,t_n) + \mathcal{O}(\Delta_t^2+\Delta_x^4), \qquad (*)$$

But we haven't shown that this also holds for the boundary nodes. τ_0^n, τ_1^n and τ_{N-1}^n are subject to the same approximation.

Let us assume that $(*)$ holds for every value of j. Hence for a certain $C>0$ independent of N and M :

$$\|\tau_j^n\|_2 \le C(\Delta_t+\Delta_x^2) \quad \text{for all } n\in\{0,\cdots,M-1\},\ j\in\{0,\cdots,N-1\}.$$

Therefore,

$$\sup_{0\le n\le M}\|\tau_h^n\|_h \le C\,(\Delta_t+\Delta_x^2),$$

and the scheme is **consistent of order** $(1,2)$.

so the scheme is first order in time and second order in space. This proves the consistency of the method.

Stability in the discrete L^2 norm. It is easy to establish that the matrix Q satisfies

$$\|Q^n\|_2 \le 1 \qquad \text{for all } n\ge 0,$$

provided that the classical CFL condition

$$0<\nu=\frac{\mu\Delta_t}{\Delta_x^2}\le\frac{1}{2} \qquad (2)$$

is fulfilled. The scheme is stable.

Convergence. Let us define the global error $e_h^n = v_h^n - \rho_h^n$. From the scheme and the consistency relation, we obtain

$$e_h^{n+1} = Qe_h^n + \Delta_t\,\tau_h^n.$$

Iterating and applying the discrete norm gives

$$\|e_h^{n+1}\|_h \le \|Q^{n+1}\|\|e_h^0\|_h + \Delta_t\sum_{m=0}^{n}\|Q^{n-m}\|\|\tau_h^m\|_h.$$

Using $\|Q^k\|\le 1$ and $e_h^0=0$,

$$\|e_h^{n+1}\|_h \le T\sup_m\|\tau_h^m\|_h \le C\,T(\Delta_t+\Delta_x^2).$$

Thus, if it is consistent, the scheme is convergent of order $\mathcal{O}(\Delta_t+\Delta_x^2)$ in discrete L^2 norm.

4 Numerical study of the convergence

Let us examine the convergence of the cellular model for three simple cases: a three-layer basin, with initial functions corresponding to the system's own functions. We will limit ourselves to cases where the coefficients $\alpha_{2k}, \beta_{2k}^i, \gamma_{2k+1}^i$ are zero if and only if $k=1$. Let us take the simple examples of initial functions corresponding to the system's eigen values. That is functions of the same form as ρ in section 2, with :

- $\alpha_{2k}=\delta_{k,1},\ \beta_{2k}^i=0,\ \gamma_{2k+1}^1=\gamma_{2k+1}^2=\gamma_{2k+1}^3=0.$
- $\alpha_{2k}=0,\ \beta_{2k}^i=\delta_{k,1},\ \gamma_{2k+1}^1=\gamma_{2k+1}^2=\gamma_{2k+1}^3=0.$
- $\alpha_{2k}=0,\ \beta_{2k}^i=\delta_{k,1},\ \gamma_3^1=\gamma_3^2=1\text{and}\gamma_3^3=-2\gamma_{2k+1}^1=\gamma_{2k+1}^2=\gamma_{2k+1}^3=0\forall k\neq 0.$

5 Mixing Optimization : Optimal Matrix

I have developed a numerical algorithm to model algal growth in a water basin. The model is based on the introduction of a source term into the discretization of the advection–diffusion equation with an upwind discretization of the advection term.

Let us first introduce the numerical scheme. Far from the paddle wheel, the PDE discretization leads to :

$$\forall n \in \{0, \cdots, M-1\}$$
$$\frac{v_j^{n+1} - v_j^n}{\Delta_t} - D\frac{v_{j+1}^n - 2v_j^n + v_{j-1}^n}{\Delta_x^2} + a\frac{v_j^n - v_{j-1}^n}{\Delta_x} = 0$$

and the numerical scheme is defined by :

$$\begin{cases} v_j^{n+1} = (\kappa+\nu)v_{j-1}^n + (1-2\nu-\kappa)v_j^n + \nu v_{j+1}^n \quad \forall j \in \{2, \cdots, N-2\} \\ v_{N-1}^{n+1} = (\kappa+\nu)v_{N-2}^n + (1-2\nu-\kappa)v_{N-1}^n + \nu P v_0^n \\ v_0^{n+1} = (\kappa+\nu)v_{N-1}^n + (1-2\nu-\kappa)P v_0^n + \nu v_1^n \\ v_1^{n+1} = (\kappa+\nu)P v_0^n + (1-2\nu-\kappa)v_1^n + \nu v_2^n \end{cases}$$

Let u now introduce the source term. Each layer i corresponds to a source term μ_i. Let $\mu = \begin{pmatrix} \mu_1 \\ \cdots \\ \mu_I \end{pmatrix} \in \mathbb{R}^I$.

$$\begin{cases} v_j^{n+1} = (\kappa+\nu)v_{j-1}^n + (1-2\nu-\kappa+\mu\Delta_t)v_j^n + \nu v_{j+1}^n \quad \forall j \in \{2, \cdots, N-2\} \\ v_{N-1}^{n+1} = (\kappa+\nu)v_{N-2}^n + (1-2\nu-\kappa+\mu\Delta_t)v_{N-1}^n + \nu P v_0^n \\ v_0^{n+1} = (\kappa+\nu)v_{N-1}^n + (1-2\nu-\kappa+\mu\Delta_t)P v_0^n + \nu v_1^n \\ v_1^{n+1} = (\kappa+\nu)P v_0^n + (1-2\nu-\kappa+\mu\Delta_t)v_1^n + \nu v_2^n \end{cases}$$

This additional term modifies the Courant–Friedrichs–Lewy (CFL) condition and, consequently, the value of the time step Δt. To converge, the numerical scheme must verify :

$$\Delta_t \leq \frac{\Delta_x^2}{2\mu + a\Delta_x}$$

5.1 introduction of a biomass source term

In this report, we present first numerical results illustrating the total biomass in the basin after a growth period of $T = 100\,\mathrm{s}$. According to [?, ?], the algae follow the Han growth model, which is of Haldane if the dynamic of the open state is at its steady state. The light intensity follows a Beer-lambert law [?]:

$$I = I_s e^{-\epsilon z}$$

With $I_s = 1500\ \mathrm{m}^{-2}\,\mu\mathrm{mol\,s}^{-1}$

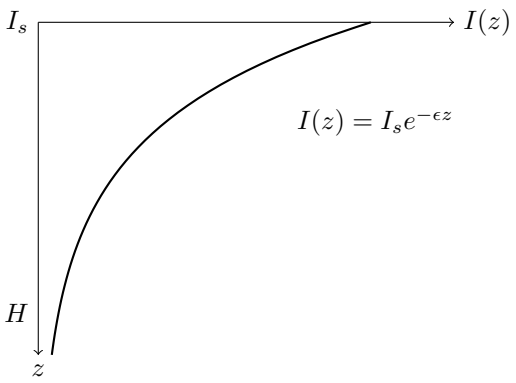


Figure 1: Exponential attenuation of light intensity with depth according to the Beer–Lambert law.

The ϵ light extinction coefficient will be computed once and for all, assuming that the light intensity at depth H is 1% of the surface light intensity I_s. We have :

$$I(H) = \frac{I_s}{100} = I_s \exp(-\epsilon H)$$

Therefore,

$$\epsilon = \frac{\ln(100)}{H} \approx 4.6$$

And for the values of the constants involved in the computing of μ :

$$\begin{aligned} k_d &= 2.99 \times 10^{-5}, \\ k_r &= 4.8 \times 10^{-5}\ \mathrm{s}^{-1}, \\ k_h &= 3.64 \times 10^{-5}, \\ \tau &= 6.849\ \mathrm{s}, \\ \sigma_H &= 2.9 \times 10^{-4}\ \mathrm{m}^2\,\mu\mathrm{mol}^{-1} \end{aligned}$$

We have

$$A = \frac{1}{\frac{k_d}{k_r}\,\tau\,(\sigma_H I)^2 + \tau\,\sigma_H I + 1}$$

And

$$\mu = A\,I\,k_h\,\sigma_H = 1.78 \cdot 10^{-4}$$

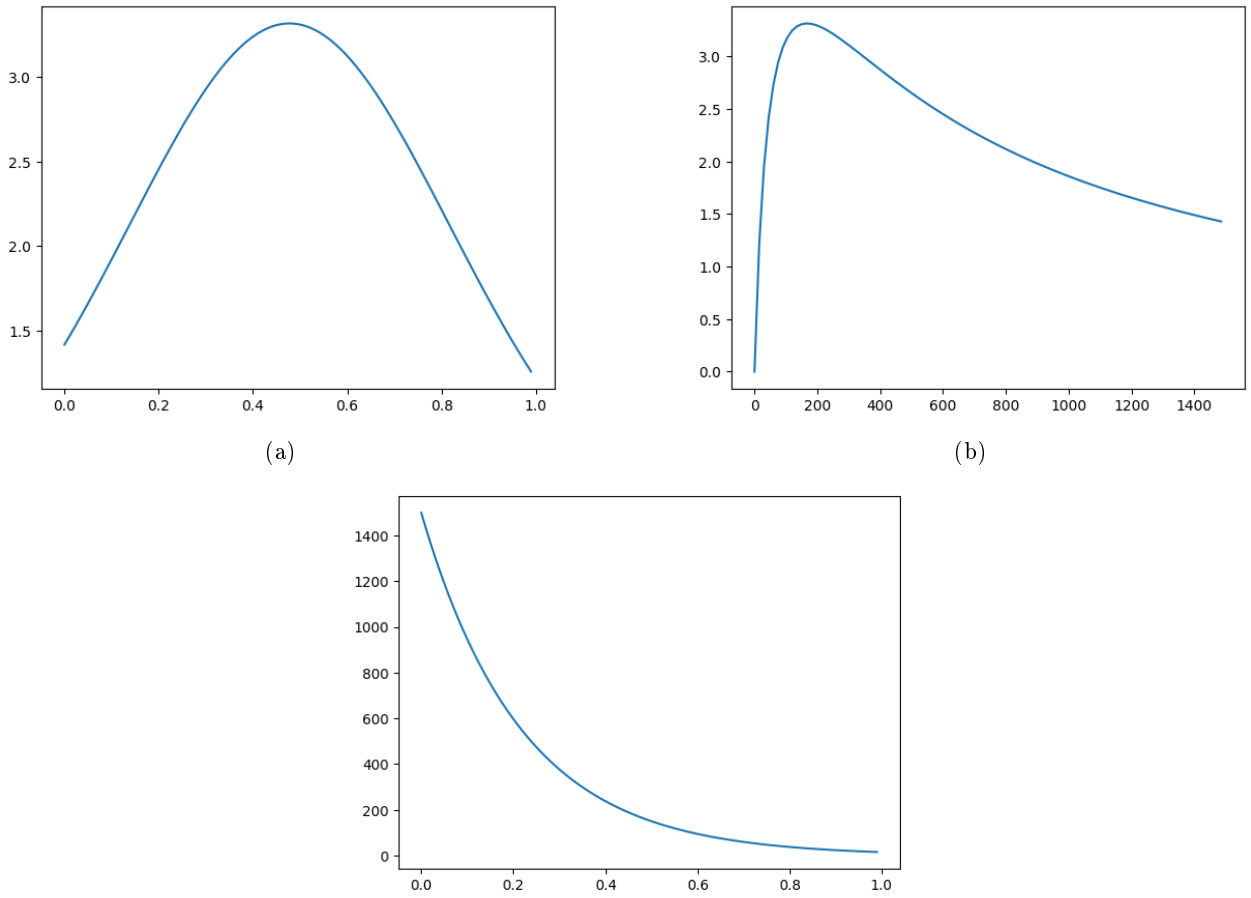


Figure 2: (a) $\mu\,(d^{-1})$ as a function of z ; (b) $\mu\,(d^{-1})$ as a function of I ; (c) I as a function of z

The simulations are performed for a single layer, and therefore without any mixing effects. The only varying parameter is the water velocity u. For a discretization of 500 points, we impose a uniform initial concentration and an initial total biomass of 50 Kg. After an hour of growth, we get the following total biomass table for different current speed values.

Current speed (m.s^{-1})	**Total biomass (kg)**
0	52.8041
0.1	52.6224
0.2	52.6222
0.3	52.6221
0.5	52.6221
1	52.6221

Table 1: Total biomass (T = 3600 s), as a function of current speed

It seems, that the biomass grows less as the current increases. One must take into account that Δ_t, the time step of the numerical scheme depends on the value of the speed u. This follows from the upwind scheme CFL condition.

6 Firt Optimization of the Mixing Matrix : The permutation case

6.1 General Principle of the Algorithm

The objective of the algorithm is to optimize a mixing matrix applied to a discretized basin, in order to maximize the total biomass produced by the system.

For a given initial configuration, a candidate mixing matrix is repeatedly applied to the system. The temporal evolution of the basin is simulated with a constant time step, for a given period T. The total biomass is then evaluated.

The algorithm explores different admissible mixing matrices (among every existing permutation matrices) and retains the one that maximizes the final total biomass.

6.2 Tested Initial Configurations

In this study, the basin consists of three vertical layers. The initial conditions are constructed by placing all the initial biomass (50 Kg) in a single layer, while the other layers start with zero biomass. Three configurations are therefore considered, corresponding to the successive activation of each layer. The idea is to choose the mixing matrix which induces a maximal biomass after an hour of growth. We will set the current speed value u to 0.3 $\mathrm{m\,s^{-1}}$. It is a rather small value, but as the CFL depends on it, a smaller speed value leads to a shorter computing time. 300 seconds of growth therefore correspond to approximately 20 laps.

6.3 Optimization Results

For each initial configuration, the optimization algorithm converges to a matrix that maximizes the final total biomass with T = 300 s.

6.3.1 Comparative Analysis

The optimization results show that the optimal mixing matrix depends strongly on the initial configuration of the basin. When biomass is localized in an inherently favorable layer (with the highest growth rate), keepig each layer unchanged is optimal (no mixing). Conversely, in less favorable configurations, vertical mixing allows biomass to be redistributed to more productive layers. We will see that the optimal matrices allocate biomass to the last layer (the deepest one). Indeed, as $\mu = A\,I\,k_h\,\sigma_H$, the μ values of each layer are the following :

Layer	μ **(s^{-1})**	**I** (μmol $m^{-2}s^{-1}$)
Layer 1	1.4198×10^{-5}	1500
Layer 2	3.0538×10^{-5}	323.72
Layer 3	2.8777×10^{-5}	69.86

Table 2: Growth rates μ for each initial layer configuration

Therefore the last layer has the biggest growth rate.

6.3.2 Configuration 1: biomass initially in layer 1

The maximal total biomass obtained is:

$$B_{\text{tot}} = 50.301 \quad \text{Kg}$$

The optimal mixing matrix is:

$$\mathbf{M}_1 = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

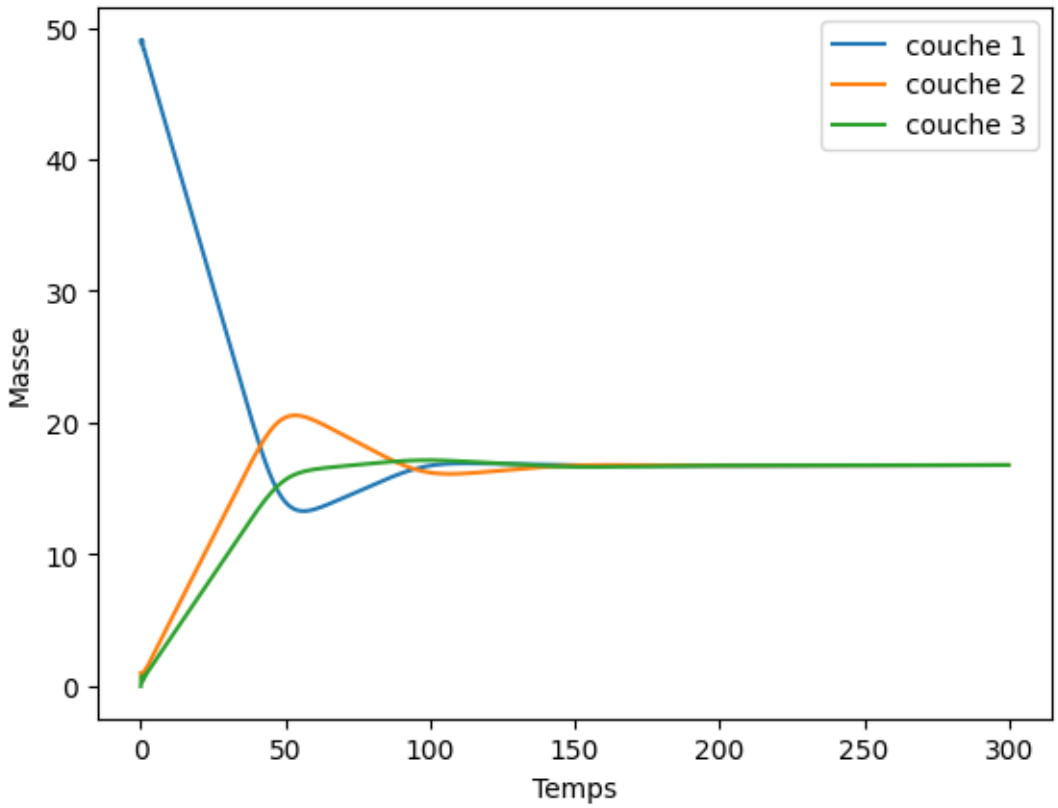


Figure 3: biomass in each layer through time

This matrix induces a permutation of layers which induces the allocation of layer 1 in position 2, corresponding to the largest growth coefficient. Here is a graphic representing the total algae mass in each layer as a function of time, with the latter permutation matrix.

6.3.3 Configuration 2: biomass initially in layer 2

The maximal total biomass obtained is:

$$B_{\text{tot}} = 50.461 \quad \text{Kg}$$

The optimal mixing matrix is:

$$\mathbf{M}_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Here, not mixing appears to be the optimal strategy.

6.3.4 Configuration 3: biomass initially in layer 3

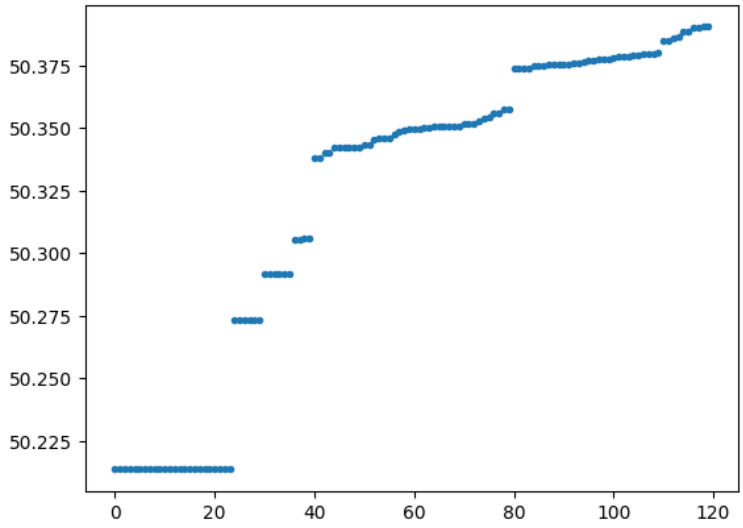
The maximal total biomass obtained is:

$$B_{\text{tot}} = 50.447 \quad \text{Kg}$$

The optimal mixing matrix is:

$$\mathbf{M}_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

This configuration corresponds to the permutation of layers 2 and 3.



6.4 other initial configurations, 5 layers

Layer	μ ($\mathbf{s^{-1}}$)	**I** (μmol $m^{-2}s^{-1}$)
Layer 1	1.4198×10^{-5}	1500
Layer 2	2.4558×10^{-5}	597.8
Layer 3	3.2373×10^{-5}	238.2
Layer 4	3.1229×10^{-5}	94.94
Layer 5	2.2153×10^{-5}	37.83

6.4.1 Configuration 1: biomass initially in layer 1

After a period T= 3600 s of growth, the total biomass obtained is:

$$B_{\text{tot}} = 50.390 \quad \text{Kg}$$

Optimal mixing matrix:

$$\mathbf{M}_1 = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

Here again, let us represent the successive biomass values associated to the tested permutation matrices :

6.4.2 Configuration 2: biomass initially in layer 2

Total biomass obtained:

$$B_{\text{tot}} = 50.443 \quad \text{Kg}$$

Optimal mixing matrix:

$$\mathbf{M}_2 = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

6.4.3 Configuration 3: biomass initially in layer 3

Total biomass obtained:

$$B_{\text{tot}} = 50.488 \quad \text{Kg}$$

Optimal mixing matrix:

$$\mathbf{M}_3 = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

6.4.4 Configuration 4: biomass initially in layer 4

Total biomass obtained:

$$B_{\text{tot}} = 50.480 \quad \text{Kg}$$

Optimal mixing matrix:

$$\mathbf{M}_4 = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

6.4.5 Configuration 5: biomass initially in layer 5

Total biomass obtained:

$$B_{\text{tot}} = 50.431 \quad \text{Kg}$$

Optimal mixing matrix:

$$\mathbf{M}_5 = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

7 Mixing Optimization : Optimal Stochastic Mixing-Matrix

Now that we have found a way to determine the best permutation matrix for the water pond, let us now focus on a more general case : the column stochastic matrices. The initial problem is indeed defined with P a $N \times N$ column-stocastic matrix. To optimize on this infinite space, we will have to use a differentiable program. We have chosen the torch python library to do so. In the update fonction, we have to convert every numpy list and array into a torch variable. This will allow us to compute torch differentiation with these variables. Furthermore, to process the matrix constraints (non negative coefficients and every column must sum 1), we will use the softmax function, in order to differentiate with respect an unconstrained matrix Q.

The objective function remains the same.

7.1 initial test

Let us first verify if the new solver is functional. In order to do so, let us consider a three layers basin, and an objective function which mixes these layers with respect to the matrix $P = \text{softmax}(Q)$ and returns the total biomass, after a pre-determined growing period. The growth coefficients will be arbitrarely set to 1 for layer 1, 2 for layer 2 and 3 for layer 3. Therefore, with an initial basin in which the total biomass is concentrated in layer 1, an optimal matrix should reallocate it in layer 3. Let us consider the results.

The optimal mixing matrix is:

$$\mathbf{P} = \begin{pmatrix} 0.003 & 0.50 & 0.26 \\ 0.007 & 0.30 & 0.10 \\ 0.99 & 0.20 & 0.64 \end{pmatrix}$$

The first layer is indeed moved to layer 3 by 99 percent.

7.2 Optimal stochastic matrix

Let us now use the original objective function, i.e the final biomass after a given growth period T. As before, the growth time remains $T = 300s$ and the initial conditions are that the total biomass is in layer 1.

7.2.1 optimizer parameters

The Adam optimizer (Adaptive Moment Estimation) combines momentum-based gradient descent with per-parameter adaptive learning rates. At each iteration t, it maintains an exponential moving average of the gradient m_t (first moment) and of the squared gradient v_t (second moment), defined as

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$

where g_t denotes the gradient of the loss with respect to the parameters. These moment estimates are corrected for initialization bias as

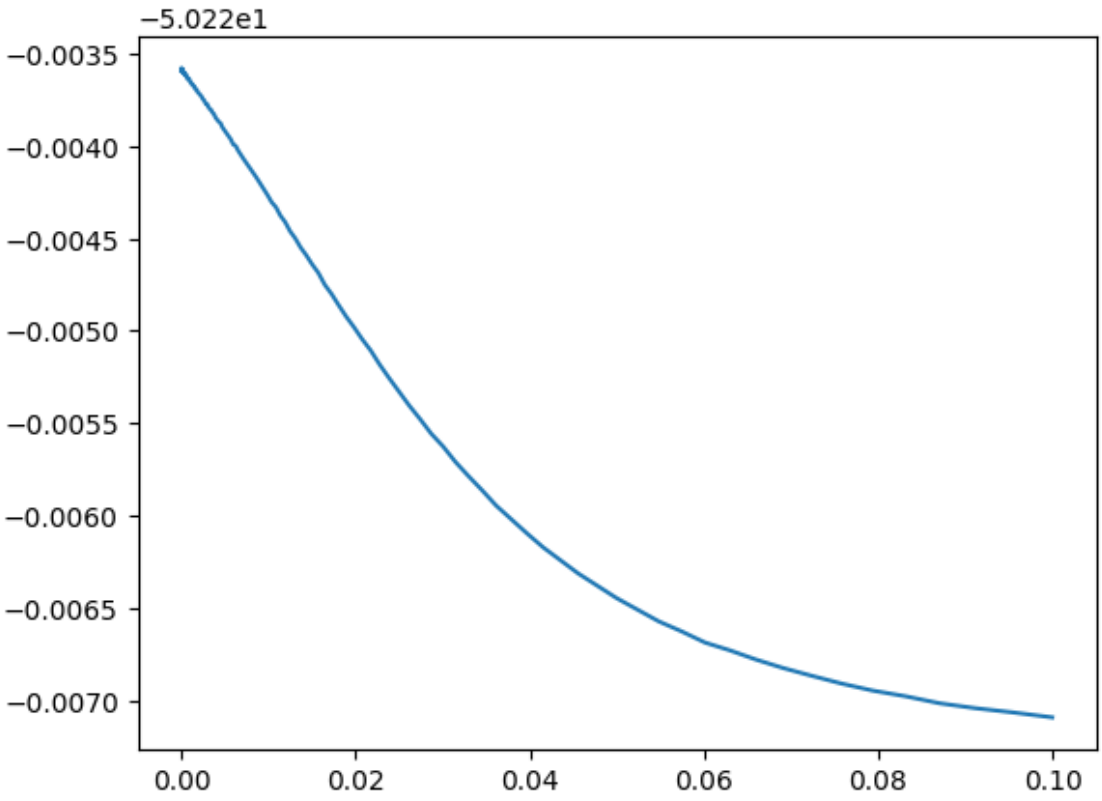
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}.$$

The parameter update is then given by

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon},$$

where α is the global learning rate, while β_1 and β_2 control the exponential decay rates of the first and second moments, respectively, governing the trade-off between stability and responsiveness of the optimization process.

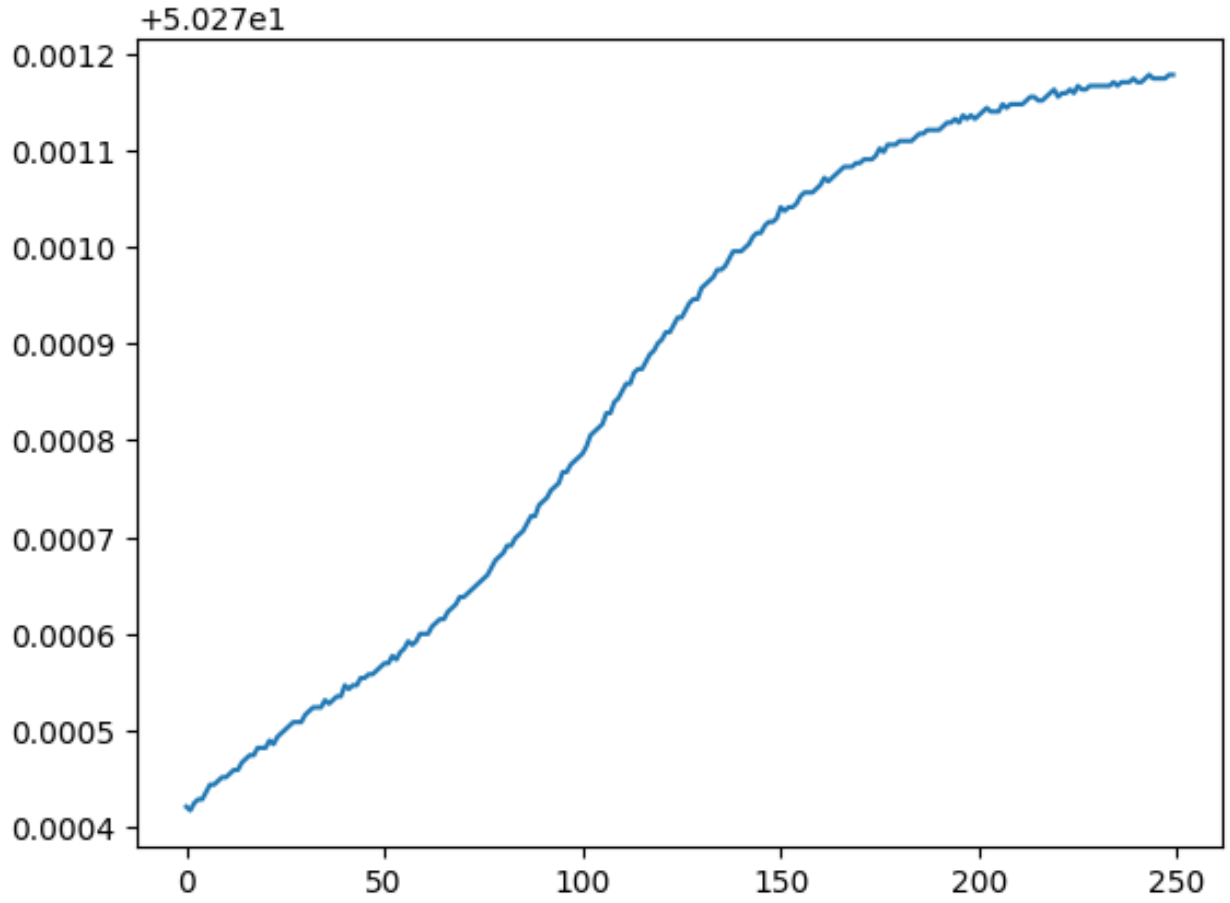
In order to choose an appropriate learning rate for the Adam optimization algorithm, we performed a learning rate finder experiment. The principle consists in running a single optimization trajectory while progressively increasing the learning rate and recording the corresponding value of the loss. The resulting curve, representing the loss as a function of the learning rate, is shown here after.



7.2.2 learning rate finder, loss vs lr

Contrary to the classical behaviour observed in many deep learning applications, the loss does not exhibit an instability or divergence for large learning rates, but instead decreases monotonically as the learning rate increases. This behaviour can be explained by the specific structure of the problem: the number of optimized parameters is small, the optimization is performed through a softmax parametrization which enforces boundedness and normalization, and the Adam optimizer adaptively rescales the update step, preventing numerical instability. As a consequence, no critical learning rate leading to divergence is observed. We therefore select a learning rate corresponding to the largest value before the curve flattens ($lr = 0.1$), which provides fast convergence while remaining numerically stable.

Once the learning rate is fixed, we analyze the convergence of the optimization by plotting the loss as a function of the iteration number.



7.2.3 loss vs iterations

The curve exhibits a clear plateau after a given number of iterations, indicating that further optimization steps do not significantly improve the objective value. Based on this observation, the optimization is stopped at this iteration count, which provides a good trade-off between computational cost and convergence accuracy.

With an initial pond in which the biomass is only in the first layer, the solver returns the following matrix :

$$\mathbf{P} = \begin{pmatrix} 0.0137 & 0.0294 & 0.0018 \\ 0.0310 & 0.2969 & 0.0044 \\ 0.9553 & 0.6737 & 0.9938 \end{pmatrix}$$

this matrix corresponds to a situation in which the final biomass is 50.301Kg. This is approximately the same result than the permutation matrix case. We can now begin the ADAM algorithm starting from the best permutation matrix and see if we obtain a better result. And indeed, the optimisation returns a new biomass of The immediate problem we face is the algorithm complexity. With $T = 300$s and a three layer pond, the solver takes about one hour to return the approximate optimal matrix.

Computational complexity. We analyze the computational complexity of the optimization procedure by first estimating the cost of a single evaluation of the objective function. The time step of the numerical scheme is given by

$$\Delta t = \mathrm{CFL}\,\frac{\Delta x^2}{2\mu + a\,\Delta x}, \quad \text{with} \quad \Delta x = \frac{L}{n},$$

where n denotes the number of spatial discretization points, μ the diffusion coefficient, and a a characteristic advection velocity. Substituting Δx yields

$$\Delta t = \mathrm{CFL}\,\frac{L^2}{n^2(2\mu + aL/n)} = \frac{\mathrm{CFL}\,L^2}{2\mu n^2 + aLn}.$$

In our setting, typical parameter values are $\mu \approx 10^{-5}$, $n \approx 50$, $0.01 < a < 0.1$, and $L \approx 5$. Under these assumptions, the term aLn dominates $2\mu n^2$, leading to the approximation

$$\Delta t \sim \frac{\mathrm{CFL}\,L}{a\,n}.$$

The objective function corresponds to the final state of a time-marching simulation over a time horizon T. The number of time steps is therefore of order $T/\Delta t$, and each time step involves $O(Nn)$ operations, where N denotes the number of layers. Consequently, the computational complexity of a single forward evaluation of the objective function is

$$\mathcal{O}\left(Nn\frac{T}{\Delta t}\right) = \mathcal{O}\left(\frac{a\,N\,n^2\,T}{\mathrm{CFL}\,L}\right).$$

Since the objective function is implemented in a differentiable manner using automatic differentiation, the backward pass requires propagating gradients through the entire time-marching scheme. The computational cost of the backward pass is of the same order as the forward pass, up to a constant factor, yielding

$$\mathcal{O}\left(\frac{a\,N\,n^2\,T}{L}\right).$$

Finally, the full optimization procedure consists of K iterations of the Adam algorithm. Each iteration requires one forward evaluation of the objective function and one backward pass, while the update of the parameter matrix $P \in \mathbb{R}^{N\times N}$ is negligible in comparison. The overall computational complexity of the `solve` algorithm is therefore

$$\mathcal{O}\left(K\,\frac{a\,N\,n^2\,T}{L}\right).$$

8 research perspectives and next steps in the work plan

As an immediate next step, we can compute this algorithm with a larger number of layers, and a longer growth period. But as the latter algorithm takes a long time to compute, this will rapidly take several days to return a result. A new field to explore could be an estimation of the algorithm complexity and a way to tackle it.
Furthermore, we could also look for other optimization methods (the Newton method for example).

References

[1] Liu-Di Lu, *Lagrangian Approaches for Modelling and Optimization of Hydrodynamic-Photosynthesis Coupling*, Sorbonne Université, LJLL, 2021

[2] Oxford reference *Beer-Lambert Law*